```
-- Databricks notebook source
-- MAGIC %md
-- MAGIC #  **SQL to PySpark Phase 3- ETL**

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **Reading  the CUSTOMERS AND SALES DATA**

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **1. Load Customer and Order Data**

-- COMMAND ----------

-- Load customer and order data from volumes into temp views
CREATE OR REPLACE TEMP VIEW customers AS
SELECT * FROM read_files('/Volumes/workspace/default/customers', format
=> 'csv', header => true);

CREATE OR REPLACE TEMP VIEW orders AS
SELECT * FROM
read_files('/Volumes/workspace/default/customers/orders.csv', format =>
'csv', header => true);

-- Count rows loaded
SELECT 'Customers' AS dataset, COUNT(*) AS row_count FROM customers
UNION ALL
SELECT 'Orders' AS dataset, COUNT(*) AS row_count FROM orders;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **2. Explore Data Schema and Structure**

-- COMMAND ----------

-- Display schema for customers
DESCRIBE customers;

-- COMMAND ----------

-- Display schema for orders
DESCRIBE orders;

-- COMMAND ----------

-- Sample customers data
SELECT * FROM customers LIMIT 5;

-- COMMAND ----------

-- Sample orders data
SELECT * FROM orders LIMIT 5;

-- COMMAND ----------
```

```
-- Count missing values per column for orders
SELECT
  SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS order_id_nulls,
  SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS
customer_id_nulls,
  SUM(CASE WHEN order_date IS NULL THEN 1 ELSE 0 END) AS
order_date_nulls,
  SUM(CASE WHEN status IS NULL THEN 1 ELSE 0 END) AS status_nulls,
  SUM(CASE WHEN total_amount IS NULL THEN 1 ELSE 0 END) AS
total_amount_nulls
FROM orders;

-- COMMAND ----------

-- Sample validated customers
SELECT * FROM customers_valid LIMIT 10;

-- COMMAND ----------

-- Sample validated orders
SELECT * FROM orders_valid LIMIT 10;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **3. Identify Missing Values**

-- COMMAND ----------

-- Count missing values per column for customers
SELECT
  SUM(CASE WHEN customer_id IS NULL THEN 1 ELSE 0 END) AS
customer_id_nulls,
  SUM(CASE WHEN first_name IS NULL THEN 1 ELSE 0 END) AS
first_name_nulls,
  SUM(CASE WHEN last_name IS NULL THEN 1 ELSE 0 END) AS
last_name_nulls,
  SUM(CASE WHEN email IS NULL THEN 1 ELSE 0 END) AS email_nulls,
  SUM(CASE WHEN phone_number IS NULL THEN 1 ELSE 0 END) AS
phone_number_nulls,
  SUM(CASE WHEN address IS NULL THEN 1 ELSE 0 END) AS address_nulls,
  SUM(CASE WHEN city IS NULL THEN 1 ELSE 0 END) AS city_nulls,
  SUM(CASE WHEN state IS NULL THEN 1 ELSE 0 END) AS state_nulls,
  SUM(CASE WHEN zip_code IS NULL THEN 1 ELSE 0 END) AS zip_code_nulls
FROM customers;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **4. Clean Data (Remove Nulls)**

-- COMMAND ----------

-- Clean data by removing rows with any null values
CREATE OR REPLACE TEMP VIEW customers_clean AS
SELECT *
FROM customers
WHERE customer_id IS NOT NULL
```

```
  AND first_name IS NOT NULL
  AND last_name IS NOT NULL
  AND email IS NOT NULL
  AND phone_number IS NOT NULL
  AND address IS NOT NULL
  AND city IS NOT NULL
  AND state IS NOT NULL
  AND zip_code IS NOT NULL;

CREATE OR REPLACE TEMP VIEW orders_clean AS
SELECT *
FROM orders
WHERE order_id IS NOT NULL
  AND customer_id IS NOT NULL
  AND order_date IS NOT NULL
  AND status IS NOT NULL
  AND total_amount IS NOT NULL;

-- Show cleaning results
SELECT
  'Customers' AS dataset,
  (SELECT COUNT(*) FROM customers) AS before_cleaning,
  (SELECT COUNT(*) FROM customers_clean) AS after_cleaning,
  (SELECT COUNT(*) FROM customers) - (SELECT COUNT(*) FROM
customers_clean) AS rows_removed
UNION ALL
SELECT
  'Orders' AS dataset,
  (SELECT COUNT(*) FROM orders) AS before_cleaning,
  (SELECT COUNT(*) FROM orders_clean) AS after_cleaning,
  (SELECT COUNT(*) FROM orders) - (SELECT COUNT(*) FROM orders_clean)
AS rows_removed;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **5. Filter and Validate Records**

-- COMMAND ----------

-- Filter customers with valid email addresses
CREATE OR REPLACE TEMP VIEW customers_valid AS
SELECT *
FROM customers_clean
WHERE email IS NOT NULL;

-- Filter orders with positive total amounts
CREATE OR REPLACE TEMP VIEW orders_valid AS
SELECT *
FROM orders_clean
WHERE CAST(total_amount AS DOUBLE) > 0;

-- Show validation counts
SELECT
  'Valid Customers' AS dataset,
  COUNT(*) AS row_count
FROM customers_valid
UNION ALL
```

```
SELECT
  'Valid Orders' AS dataset,
  COUNT(*) AS row_count
FROM orders_valid;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **6. Read JSON and Parquet Sample Files**

-- COMMAND ----------

-- Sample starter code - read CSV file
CREATE OR REPLACE TEMP VIEW df AS
SELECT * FROM csv.`/Volumes/workspace/default/customers/customers.csv`;

-- Display CSV data sample
SELECT * FROM df LIMIT 20;

-- Show row count after cleaning (removing nulls)
SELECT COUNT(*) AS rows_after_cleaning
FROM df
WHERE customer_id IS NOT NULL
  AND first_name IS NOT NULL
  AND last_name IS NOT NULL
  AND email IS NOT NULL;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **Business Pipeline 1: Daily Sales Analysis**

-- COMMAND ----------

-- Business Pipeline 1: Daily Sales Analysis
-- Read sales/orders data -> clean nulls -> calculate daily sales
SELECT
  TO_DATE(order_date) AS order_date,
  SUM(CAST(total_amount AS DOUBLE)) AS total_sales
FROM orders
WHERE order_id IS NOT NULL
  AND customer_id IS NOT NULL
  AND order_date IS NOT NULL
  AND total_amount IS NOT NULL
GROUP BY TO_DATE(order_date)
ORDER BY order_date;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **Business Pipeline 2: City-Wise Revenue Analysis**

-- COMMAND ----------

-- Business Pipeline 2: City-Wise Revenue Analysis
-- Join customers with orders and calculate revenue by city
SELECT
  c.city,
```

```
  c.state,
  SUM(CAST(o.total_amount AS DOUBLE)) AS total_revenue
FROM customers_clean c
INNER JOIN orders_clean o
  ON c.customer_id = o.customer_id
GROUP BY c.city, c.state
ORDER BY total_revenue DESC;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **Business Pipeline 3: Identify Repeat Customers**

-- COMMAND ----------

-- Business Pipeline 3: Identify Repeat Customers
-- Find customers with more than 2 orders
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  c.email,
  c.city,
  COUNT(o.order_id) AS order_count
FROM orders_clean o
INNER JOIN customers_clean c
  ON o.customer_id = c.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email, c.city
HAVING COUNT(o.order_id) > 2
ORDER BY order_count DESC;

-- COMMAND ----------

-- Count total repeat customers
SELECT COUNT(DISTINCT customer_id) AS total_repeat_customers
FROM (
  SELECT customer_id, COUNT(order_id) AS order_count
  FROM orders_clean
  GROUP BY customer_id
  HAVING COUNT(order_id) > 2
);

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **Business Pipeline 4: Highest Spending Customer per City**

-- COMMAND ----------

-- Business Pipeline 4: Highest Spending Customer per City
-- Use window function to find top spender in each city
WITH customer_spend AS (
  SELECT
    c.customer_id,
    c.first_name,
    c.last_name,
    c.email,
    c.city,
```

```
    c.state,
    SUM(CAST(o.total_amount AS DOUBLE)) AS total_spend
  FROM customers_clean c
  INNER JOIN orders_clean o
    ON c.customer_id = o.customer_id
  GROUP BY c.customer_id, c.first_name, c.last_name, c.email, c.city,
c.state
),
ranked_customers AS (
  SELECT
    *,
    ROW_NUMBER() OVER (PARTITION BY city ORDER BY total_spend DESC) AS
rank
  FROM customer_spend
)
SELECT
  city,
  state,
  customer_id,
  first_name,
  last_name,
  email,
  total_spend
FROM ranked_customers
WHERE rank = 1
ORDER BY total_spend DESC;

-- COMMAND ----------

-- MAGIC %md
-- MAGIC **Business Pipeline 5: Final Reporting Table**

-- COMMAND ----------

-- Business Pipeline 5: Final Reporting Table
-- Build comprehensive customer reporting table
CREATE OR REPLACE TEMP VIEW final_report AS
SELECT
  c.customer_id,
  c.first_name,
  c.last_name,
  c.email,
  c.city,
  c.state,
  SUM(CAST(o.total_amount AS DOUBLE)) AS total_spend,
  COUNT(o.order_id) AS order_count
FROM customers_clean c
INNER JOIN orders_clean o
  ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.first_name, c.last_name, c.email, c.city,
c.state
ORDER BY total_spend DESC;

-- Display final report
SELECT * FROM final_report;

-- Summary statistics
SELECT
```

```
  SUM(total_spend) AS grand_total_revenue,
  SUM(order_count) AS total_orders,
  COUNT(customer_id) AS total_customers
FROM final_report;

-- COMMAND ----------

-- Count total customers in final report
SELECT COUNT(*) AS total_customers_with_orders FROM final_report;

-- COMMAND ----------



-- COMMAND ----------

-- MAGIC %md
-- MAGIC
-- MAGIC
```